

01 — ETL with PostgreSQL

Table of Contents

1. [Learning Objectives](#)
 2. [What is ETL?](#)
 3. [PostgreSQL as ETL Source](#)
 4. [PostgreSQL as ETL Sink](#)
 5. [PostgreSQL as Transformation Layer](#)
 6. [COPY Command Deep Dive](#)
 7. [Bulk Loading Strategies](#)
 8. [Foreign Data Wrappers for ETL](#)
 9. [Common Mistakes](#)
 10. [Best Practices](#)
 11. [Performance Considerations](#)
 12. [Interview Questions & Answers](#)
 13. [Exercises & Solutions](#)
 14. [Real-World Scenarios](#)
 15. [Cross-References](#)
-

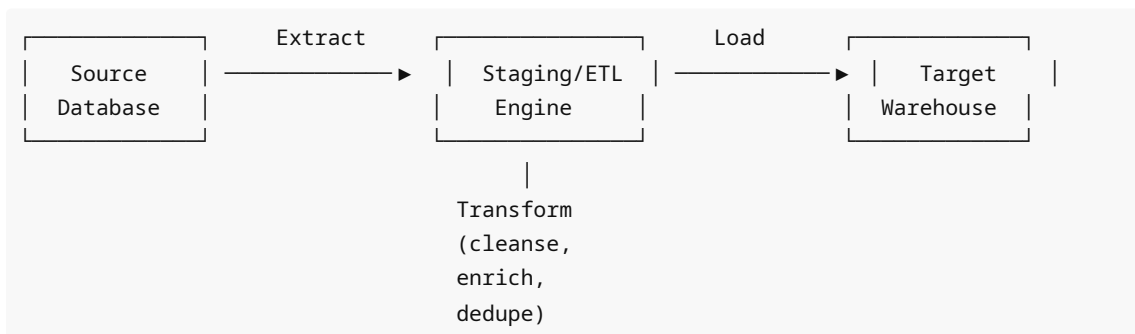
Learning Objectives

By the end of this module you will be able to:

- Design ETL pipelines with PostgreSQL as source, sink, or transformation engine
 - Use the `COPY` command for high-throughput bulk loads
 - Apply staging table patterns for reliable data ingestion
 - Tune PostgreSQL parameters for bulk operations
 - Implement error-handling and audit trails in ETL pipelines
-

What is ETL?

Extract → Transform → Load is the classic pattern for moving data between systems.



PostgreSQL can occupy **any** or **all** of these roles simultaneously.

PostgreSQL as ETL Source

Full Extract

```
-- Export entire table to CSV for downstream processing
COPY orders TO '/tmp/orders_full.csv' WITH (
  FORMAT CSV,
  HEADER TRUE,
  DELIMITER ',',
  NULL '\N',
  ENCODING 'UTF8'
);
```

Incremental Extract using watermark

```
-- Extract only rows modified since last run
COPY (
  SELECT order_id, customer_id, total_amount, status, updated_at
  FROM orders
  WHERE updated_at > '2024-01-15 00:00:00'::timestampz
  ORDER BY updated_at
) TO '/tmp/orders_delta.csv' WITH (FORMAT CSV, HEADER TRUE);
```

Extract via `psql` (pipeline-friendly)

```
psql -h db.example.com -U etl_user -d production \
-c "\COPY (SELECT * FROM events WHERE event_date = CURRENT_DATE - 1) \
  TO STDOUT WITH CSV HEADER" \
| gzip > /data/events_$(date +%Y%m%d).csv.gz
```

Logical Replication Slot as Streaming Source

```
-- Create a replication slot for CDC-style extraction
SELECT pg_create_logical_replication_slot('etl_slot', 'pgoutput');

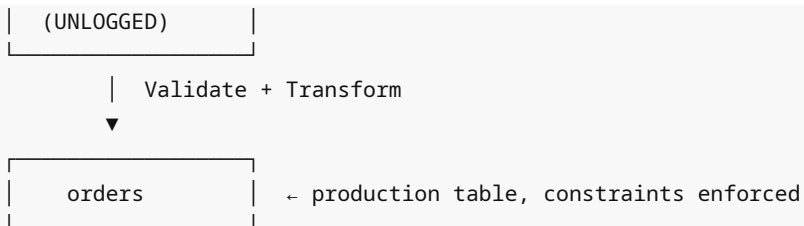
-- Peek at changes without consuming them
SELECT * FROM pg_logical_slot_peek_changes('etl_slot', NULL, NULL);

-- Consume changes
SELECT * FROM pg_logical_slot_get_changes('etl_slot', NULL, NULL);
```

PostgreSQL as ETL Sink

Staging Table Pattern

```
Raw CSV / API / Stream
  |
  ▼
┌───────────┐
│ staging_orders │ ← raw, unvalidated, nullable columns
```



```
-- Step 1: Create staging table (UNLOGGED = faster, no WAL overhead)
CREATE UNLOGGED TABLE staging_orders (
    raw_id          TEXT,
    raw_customer    TEXT,
    raw_amount      TEXT,
    raw_date        TEXT,
    load_timestamp  TIMESTAMPTZ DEFAULT NOW(),
    load_batch_id   UUID        DEFAULT gen_random_uuid()
);

-- Step 2: Bulk load raw data
COPY staging_orders (raw_id, raw_customer, raw_amount, raw_date)
FROM '/tmp/orders.csv'
WITH (FORMAT CSV, HEADER TRUE, NULL '');

-- Step 3: Transform + validate into production table
INSERT INTO orders (order_id, customer_id, total_amount, order_date)
SELECT
    raw_id::BIGINT,
    raw_customer::INT,
    raw_amount::NUMERIC(12,2),
    raw_date::DATE
FROM staging_orders
WHERE raw_id ~ '^\\d+$'
    AND raw_amount ~ '^\\d+(\\.\\d{1,2})?$',
    AND raw_date ~ '^\\d{4}-\\d{2}-\\d{2}$'
ON CONFLICT (order_id) DO UPDATE
    SET total_amount = EXCLUDED.total_amount,
        order_date = EXCLUDED.order_date;

-- Step 4: Log rejected rows
INSERT INTO etl_rejected_rows (source_table, raw_data, rejection_reason, batch_id)
SELECT
    'orders',
    row_to_json(s)::TEXT,
    CASE
        WHEN raw_id !~ '^\\d+$' THEN 'invalid order_id'
        WHEN raw_amount !~ '^\\d+(\\.\\d{1,2})?$', THEN 'invalid amount'
        ELSE 'invalid date'
    END,
    load_batch_id
FROM staging_orders s
WHERE raw_id !~ '^\\d+$'
```

```

OR raw_amount !~ '^\\d+(\\.\\d{1,2})?$',
OR raw_date !~ '^\\d{4}-\\d{2}-\\d{2}$';

-- Step 5: Truncate staging
TRUNCATE staging_orders;

```

PostgreSQL as Transformation Layer

PostgreSQL's SQL engine is powerful enough to perform complex transformations without extracting data to Python/Spark:

```

-- Transformation: Normalize addresses, compute derived fields, deduplicate
WITH ranked_orders AS (
  SELECT *,
         ROW_NUMBER() OVER (
           PARTITION BY customer_id, DATE_TRUNC('day', created_at)
           ORDER BY created_at DESC
         ) AS rn
  FROM staging_orders_raw
),
cleaned AS (
  SELECT
    order_id::BIGINT                AS order_id,
    TRIM(UPPER(customer_email))     AS customer_email,
    ROUND(raw_amount::NUMERIC, 2)   AS amount,
    created_at AT TIME ZONE 'UTC'    AS created_at_utc,
    REGEXP_REPLACE(phone, '^[0-9]', '', 'g') AS phone_clean,
    COALESCE(NULLIF(country_code, ''), 'US') AS country_code
  FROM ranked_orders
  WHERE rn = 1 -- deduplicate
),
enriched AS (
  SELECT c.*,
         cu.segment,
         cu.lifetime_value
  FROM cleaned c
  LEFT JOIN customers cu USING (customer_email)
)
INSERT INTO orders_warehouse
SELECT * FROM enriched;

```

COPY Command Deep Dive

Syntax Reference

```

-- Server-side COPY (requires superuser or pg_read_server_files role)
COPY table_name [ (column_list) ]
FROM { 'filename' | PROGRAM 'command' | STDIN }
WITH (

```

```

FORMAT      { TEXT | CSV | BINARY },
DELIMITER    'char',
NULL         'string',
HEADER       { TRUE | FALSE },
QUOTE        'char',          -- CSV only
ESCAPE       'char',          -- CSV only
FORCE_NULL   (col1, col2),    -- treat empty string as NULL
ENCODING     'encoding'
);

-- Client-side \COPY (runs as OS user, no superuser needed)
\COPY table_name FROM 'local_file.csv' WITH (FORMAT CSV, HEADER);

```

COPY FROM PROGRAM (dynamic pipelines)

```

-- Decompress on the fly while loading
COPY events FROM PROGRAM 'gunzip -c /data/events_20240115.csv.gz'
WITH (FORMAT CSV, HEADER TRUE);

-- Fetch from S3 using AWS CLI
COPY s3_data FROM PROGRAM
    'aws s3 cp s3://my-bucket/data.csv - 2>/dev/null'
WITH (FORMAT CSV, HEADER TRUE);

```

Binary COPY (fastest, no parsing overhead)

```

-- Export binary
COPY orders TO '/tmp/orders.bin' WITH (FORMAT BINARY);

-- Import binary (same schema required)
COPY orders_archive FROM '/tmp/orders.bin' WITH (FORMAT BINARY);

```

COPY performance benchmark

Method	1M rows (approx)
INSERT row-by-row	~120 seconds
Multi-row INSERT	~20 seconds
COPY TEXT	~5 seconds
COPY CSV	~6 seconds
COPY BINARY	~3 seconds

Bulk Loading Strategies

Pre-load optimizations

```

-- 1. Disable triggers temporarily (use with caution)
ALTER TABLE orders DISABLE TRIGGER ALL;

-- 2. Drop indexes before bulk load, rebuild after
DROP INDEX CONCURRENTLY idx_orders_customer_id;
DROP INDEX CONCURRENTLY idx_orders_created_at;

-- 3. Set fill factor high for append-only loads
ALTER TABLE orders SET (fillfactor = 100);

-- 4. Bulk load
COPY orders FROM '/tmp/orders_bulk.csv' WITH (FORMAT CSV, HEADER);

-- 5. Rebuild indexes
CREATE INDEX CONCURRENTLY idx_orders_customer_id ON orders(customer_id);
CREATE INDEX CONCURRENTLY idx_orders_created_at ON orders(created_at);

-- 6. Re-enable triggers
ALTER TABLE orders ENABLE TRIGGER ALL;

-- 7. Update statistics
ANALYZE orders;

```

Session-level bulk load tuning

```

-- Run these BEFORE your COPY session
SET maintenance_work_mem = '1GB';
SET synchronous_commit = OFF;           -- risky but fast for idempotent loads
SET wal_level = minimal;                 -- only if no replication

```

Partitioned table bulk load

```

-- Load directly into partition for speed
COPY orders_2024_01 FROM '/tmp/orders_jan.csv' WITH (FORMAT CSV, HEADER);
COPY orders_2024_02 FROM '/tmp/orders_feb.csv' WITH (FORMAT CSV, HEADER);
-- Parent table routes via partition routing, but direct is faster

```

Parallel COPY with multiple workers

```

# Split large file and load in parallel
split -l 1000000 orders_huge.csv orders_part_
for f in orders_part_*; do
    psql -d mydb -c "\COPY orders FROM '$f' CSV HEADER" &
done
wait
echo "All parts loaded"

```

Foreign Data Wrappers for ETL

```
-- Install postgres_fdw to pull from another PostgreSQL
CREATE EXTENSION postgres_fdw;

CREATE SERVER source_db
  FOREIGN DATA WRAPPER postgres_fdw
  OPTIONS (host 'source.internal', port '5432', dbname 'production');

CREATE USER MAPPING FOR etl_user
  SERVER source_db
  OPTIONS (user 'readonly_user', password 'secret');

-- Map remote table locally
CREATE FOREIGN TABLE remote_orders (
  order_id    BIGINT,
  customer_id INT,
  amount      NUMERIC,
  created_at  TIMESTAMPTZ
)
SERVER source_db
OPTIONS (schema_name 'public', table_name 'orders');

-- ETL: pull remote data into local warehouse
INSERT INTO orders_warehouse
SELECT * FROM remote_orders
WHERE created_at >= CURRENT_DATE - INTERVAL '1 day';
```

Common Mistakes

Mistake	Consequence	Fix
Loading directly into production table	Partial load on failure corrupts data	Always use a staging table
Forgetting ANALYZE after bulk load	Planner uses stale statistics	Run ANALYZE post-load
Using synchronous_commit=OFF without idempotency	Data loss on crash	Only use with idempotent loads
Single large transaction for COPY	Bloats WAL, long rollback	Use COPY in smaller batches
Not dropping indexes before bulk insert	10x slower load	Drop and rebuild after
Leaving UNLOGGED tables in production	Data loss on crash	Log to production tables

Best Practices

1. **Always stage first** — load raw data, then transform into target.

2. **Capture rejected rows** — never silently discard bad records.
 3. **Record batch metadata** — batch ID, row count, load time, source file.
 4. **Use ON CONFLICT for idempotency** — re-runnable ETL is safe ETL.
 5. **Prefer COPY over INSERT** — at least 10x faster for bulk.
 6. **Partition target tables** — range partitioning on date columns speeds up loads and queries.
 7. **Run VACUUM ANALYZE** after large deletes or updates in staging.
-

Performance Considerations

```
-- postgresql.conf tuning for ETL workloads
-- (apply during bulk load windows, revert for OLTP)

checkpoint_completion_target = 0.9
wal_buffers                  = 64MB
max_wal_size                 = 4GB
maintenance_work_mem        = 2GB      -- for index builds
work_mem                     = 256MB   -- for sort-heavy transforms
synchronous_commit           = off      -- if data is re-loadable
checkpoint_timeout           = 15min
```

```
-- Monitor bulk load progress
SELECT
    pid,
    now() - pg_stat_activity.query_start AS duration,
    query,
    state
FROM pg_stat_activity
WHERE query ILIKE '%COPY%'
    AND state != 'idle';

-- Check table bloat after ETL
SELECT
    schemaname,
    tablename,
    pg_size_pretty(pg_total_relation_size(schemaname||'.'||tablename)) AS total_size
FROM pg_tables
WHERE schemaname = 'public'
ORDER BY pg_total_relation_size(schemaname||'.'||tablename) DESC;
```

Interview Questions & Answers

Q1: What is the difference between COPY and \COPY ?

COPY runs on the server — the file must exist on the database server and the user needs pg_read_server_files or superuser privilege. \COPY is a psql meta-command that reads files from the client machine, making it safe for non-superusers and suitable for CI/CD pipelines.

Q2: Why use a staging table instead of loading directly into the production table?

Staging isolates bad data, allows transformation before constraints are enforced, enables partial-failure recovery, and keeps the production table consistent throughout the load window.

Q3: How would you make an ETL pipeline idempotent?

Use `INSERT ... ON CONFLICT (pk) DO UPDATE (upsert)`, load into a staging table keyed by batch ID, or truncate-and-reload partitions atomically with partition detach/attach.

Q4: What is `synchronous_commit = off` and when should you use it?

It allows the server to acknowledge commits before WAL is flushed to disk, improving throughput. Safe only when data can be re-loaded on crash (idempotent pipelines). Never use for transactional data where loss is unacceptable.

Q5: How does COPY BINARY differ from COPY CSV?

Binary format skips text parsing, making it faster but non-portable across different PostgreSQL versions or OS byte orders. CSV is portable and human-readable but slower due to text parsing.

Q6: What happens if a COPY command encounters a malformed row?

The entire `COPY` command fails and is rolled back. Workaround: use `COPY FROM PROGRAM` to preprocess data, or use `file_fdw` with error handling, or validate in a staging table.

Q7: How do Foreign Data Wrappers fit into an ETL pipeline?

FDWs present remote data sources as local tables. ETL can then use plain SQL (`INSERT INTO local SELECT * FROM foreign_table`) without writing application code, keeping the pipeline inside the database.

Q8: What PostgreSQL features replace the Transform step in traditional ETL tools?

CTEs, window functions, `FILTER`, `JSONB` operators, regex functions, custom aggregate functions, and `plpgsql` stored procedures all allow complex transformations directly in SQL.

Q9: How would you monitor an in-progress COPY operation?

Query `pg_stat_activity` for the COPY query, monitor `pg_stat_progress_copy` (PostgreSQL 14+), and check `pg_stat_io` for I/O throughput.

Q10: Explain the staging → production ETL pattern for a slowly changing dimension.

Load raw data into staging. Join with the current dimension table to detect changes. Expire old rows (set `valid_to = NOW()`), insert new rows with `valid_from = NOW()`, and leave unchanged rows as-is. Wrap in a transaction.

Exercises & Solutions

Exercise 1: Build a staging pipeline

Task: Create a staging table for a `products` CSV, load it, reject invalid rows, and move clean data to production.

Solution:

```
CREATE UNLOGGED TABLE stg_products (  
    raw_sku      TEXT, raw_name TEXT, raw_price TEXT,
```

```

        raw_stock      TEXT, loaded_at TIMESTAMPTZ DEFAULT NOW()
    );

\COPY stg_products(raw_sku, raw_name, raw_price, raw_stock)
FROM 'products.csv' CSV HEADER;

INSERT INTO products (sku, name, price, stock)
SELECT raw_sku, raw_name, raw_price::NUMERIC, raw_stock::INT
FROM stg_products
WHERE raw_price ~ '^\\d+\\.?\\d*$' AND raw_stock ~ '^\\d+$'
ON CONFLICT (sku) DO UPDATE
    SET name=EXCLUDED.name, price=EXCLUDED.price, stock=EXCLUDED.stock;

TRUNCATE stg_products;

```

Exercise 2: Parallel COPY script

Task: Write a bash script to split a 10M row CSV and load in parallel.

Solution:

```

#!/bin/bash
FILE="$1"
TABLE="$2"
CHUNKS=8
LINES=$(wc -l < "$FILE")
PER_CHUNK=$((LINES / CHUNKS))

# Keep header
HEADER=$(head -1 "$FILE")
tail -n +2 "$FILE" | split -l "$PER_CHUNK" - /tmp/chunk_

for f in /tmp/chunk_*; do
    (echo "$HEADER"; cat "$f") > "${f}.csv"
    psql -d mydb -c "\COPY $TABLE FROM '${f}.csv' CSV HEADER" &
done
wait
rm /tmp/chunk_*
echo "Done"

```

Real-World Scenarios

Scenario: Nightly order sync from OLTP to warehouse An e-commerce platform has 50M order rows in PostgreSQL OLTP. Every night, new and updated orders must land in a Redshift warehouse.

Solution steps:

1. Extract delta rows using `updated_at > last_watermark` via `COPY TO STDOUT` piped to `gzip`.
2. Upload to S3 with `aws s3 cp`.
3. Trigger Redshift `COPY FROM S3`.
4. Update watermark table on success.

Scenario: Real-time product catalog refresh A vendor pushes a 500K row CSV every 6 hours. We need to upsert into `products` without downtime.

Solution: COPY to UNLOGGED staging → upsert into production with `ON CONFLICT DO UPDATE` → swap stats with `ANALYZE` .

Cross-References

- `02_elt_patterns.md` — ELT: transformation *inside* PostgreSQL
- `03_cdc_change_data_capture.md` — streaming extraction via WAL
- `05_batch_processing.md` — chunked update and upsert patterns
- `06_incremental_loading.md` — watermark and `deleted_at` strategies
- `08_postgresql_and_airflow.md` — orchestrating ETL with Airflow